

UNIVERSITÀ DEGLI STUDI DI MILANO
FACOLTÀ DI SCIENZE E TECNOLOGIE

DIPARTIMENTO DI INFORMATICA
“GIOVANNI DEGLI ANTONI”



LAUREA MAGISTRALE IN
SICUREZZA INFORMATICA

TODO: TITOLO DELLA TESI

Autore: Enea Manzi
Matricola: 65935A

Relatore: Prof. Marco Anisetti
Correlatore: Prof. Claudio Agostino Ardagna

A.A. 2025-2026

Abstract

Ringraziamenti

Indice

Abstract	ii
Ringraziamenti	iii
Elenco delle figure	v
Elenco delle tabelle	vi
Elenco dei codici	vii
1 Introduzione	1
1.1 Motivazione e gaps	1
1.2 Obiettivi	1
1.3 Struttura della tesi	1
2 Background e Stato dell'Arte	2
2.1 Esempio di sezione	2
3 Metodologia, Architettura e Principi di Design	3
3.1 Principi Fondamentali: il Perché delle Scelte Architettureali	3
3.2 Architettura dell'Assessment Engine	6
3.2.1 Flusso di Dipendenze Monodirezionale (P02)	6
3.2.2 DAG-Based Scheduling e Orchestrazione Deterministica (P06) . .	7
3.2.3 Split State e Immutabilità (P04)	7
3.2.4 Gateway-Agnostic Adapter (P18)	8
3.2.5 Streaming Evidence Store (P11)	9
3.2.6 Zero External State Dependency (P34)	10
3.3 Domini di Sicurezza e Box Gradient Applicato	10
4 Implementazione	13
5 Validazione sperimentale e risultati	14
5.1 Setup sperimentale	14
5.2 Risultati	14
6 Discussione e Sviluppi Futuri	15

Indice

7 Conclusioni	16
7.1 Conclusioni	16
7.2 Lavori futuri	16
Bibliografia	18

Elenco delle figure

Elenco delle tabelle

Tabella 3.1	Tassonomia delle 39 proprietà architetture di APIGuard Assurance, organizzate per categoria funzionale.	4
-------------	--	---

Elenco dei codici

Capitolo 1

Introduzione

1.1 Motivazione e gaps

1.2 Obiettivi

1.3 Struttura della tesi

Capitolo 2

Background e Stato dell'Arte

2.1 Esempio di sezione

Come discusso in letteratura [?, ?], ...

Capitolo 3

Metodologia, Architettura e Principi di Design

Il progetto di APIGuard Assurance è formalizzato in 39 proprietà architetturali, organizzate nella Tabella 3.1 in sette categorie omogenee. La tabella non è una classificazione decorativa: costituisce la mappa di navigazione dell'intero sistema, l'indice che consente al lettore di rintracciare la trattazione di ciascun aspetto progettuale nel capitolo appropriato. Le proprietà che rispondono alla domanda *perché il sistema è costruito in questa forma* vengono discusse nelle Sezioni 3.1 e ??; quelle che descrivono *come i componenti realizzano tale forma* trovano trattazione nel Capitolo 3. In ogni caso, nessuna proprietà viene omessa: la tabella garantisce tracciabilità completa tra decisioni progettuali e loro realizzazione in codice.

3.1 Principi Fondamentali: il Perché delle Scelte Architetturali

Quattro proprietà strutturano la motivazione profonda del sistema: non descrivono cosa fanno i componenti, ma giustificano perché esiste questa forma architeturale e non un'altra. Non si tratta di assiomi indipendenti, ma di una catena causale: la prima stabilisce cosa il sistema conosce del target, la seconda come usa quella conoscenza, la terza come traduce quella conoscenza in comportamento deterministico, la quarta cosa quella determinatezza garantisce sul piano scientifico. Comprenderle nell'ordine in cui si condizionano a vicenda è il modo più diretto per capire perché l'intero sistema è costruito come illustrato nel Capitolo 3.

Che qualsiasi API REST documentata sia testabile senza modifiche al codice è la conseguenza diretta dell'**agnosticismo applicativo** (P01 (CHANGE-ME)): il tool non contiene nessun riferimento hardcoded a path, strutture dati o comportamenti di un'ap-

Tabella 3.1: Tassonomia delle 39 proprietà architetture di APIGuard Assurance, organizzate per categoria funzionale.

Categoria	Proprietà
Architettura & Design	P01 (CHANGE-ME), P02 (CHANGE-ME), P04 (CHANGE-ME), P07 (CHANGE-ME), P08 (CHANGE-ME), P18 (CHANGE-ME)
Estensibilità & Manutenibilità	P05 (CHANGE-ME), P06 (CHANGE-ME), P09 (CHANGE-ME), P10 (CHANGE-ME), P19 (CHANGE-ME), P24 (CHANGE-ME), P30 (CHANGE-ME), P32 (CHANGE-ME)
Config & Riproducibilità	P03 (CHANGE-ME), P20 (CHANGE-ME), P21 (CHANGE-ME), P28 (CHANGE-ME), P34 (CHANGE-ME)
Robustezza & Sicurezza	P11 (CHANGE-ME), P12 (CHANGE-ME), P13 (CHANGE-ME), P14 (CHANGE-ME), P15 (CHANGE-ME), P16 (CHANGE-ME), P31 (CHANGE-ME), P33 (CHANGE-ME), P35 (CHANGE-ME)
Qualità & Osservabilità	P22 (CHANGE-ME), P23 (CHANGE-ME), P26 (CHANGE-ME), P27 (CHANGE-ME), P29 (CHANGE-ME)
CI/CD & DevEx	P17 (CHANGE-ME), P25 (CHANGE-ME), P39 (CHANGE-ME)
Packaging & Deployment	P36 (CHANGE-ME), P37 (CHANGE-ME), P38 (CHANGE-ME)

plicazione specifica. Tutta la conoscenza del target è derivata a runtime da due sole sorgenti, ossia il file `config.yaml` per i parametri operativi e la specifica OpenAPI per la topologia degli endpoint. Un test che consulta la superficie di attacco e non trova endpoint applicabili alle proprie condizioni restituisce uno stato di `SKIP`, non un errore: lo `SKIP` è la conseguenza prevista di uno scope legittimamente vuoto, distinta dall'`ERROR` che segnalerebbe una condizione imprevista nell'infrastruttura del tool stesso. Questa distinzione semantica non è accessoria; è la forma in cui l'agnosticismo si manifesta nel comportamento osservabile del sistema.

Il costo accettato è la dipendenza da una specifica valida. Un tool che non può verificare la correttezza del proprio oracolo è un tool che può produrre assessment parziali su specifica errata senza diagnosticarlo. Questa tensione costituisce il limite strutturale discusso nella Sezione ??, dove si argomenta che non si tratta di un difetto da correggere ma di un problema aperto nel testing contract-driven già documentato in letteratura [?].

L'agnosticismo stabilisce la sorgente della conoscenza, ma non dice come quella conoscenza viene usata. **P20 (CHANGE-ME)** colma questo spazio: la specifica OpenAPI non è documentazione destinata al tester umano, ma il vincolo contrattuale che guida l'intera pipeline. Non descrive il target, lo vincola. A partire da questa specifica il sistema costruisce payload strutturalmente validi, discrimina gli endpoint documentati dalle Shadow API, e valuta le risposte rispetto a ciò che il contratto prevede. Uno scanner HTTP tradizionale che genera payload casuali affronta un muro combinatorio: la maggioranza delle richieste viene rifiutata con `HTTP 400` prima di raggiungere la logica applicativa che si intende sondare, perché i tipi, i formati e i vincoli dichiarati nello schema non sono rispettati. L'uso della specifica come oracolo formale abbatte questo muro: ogni richiesta è costruita nel rispetto dei vincoli dichiarati, spostando il risultato dal dominio della sintassi a quello della semantica.

Ne consegue un paradosso che vale la pena di enunciare con precisione: più il sistema si affida alla specifica come oracolo, più il suo comportamento dipende dalla correttezza di quell'oracolo. Una specifica obsoleta o deliberatamente incompleta produce un assessment parziale senza che il sistema possa rilevarne l'incoerenza. Non è un difetto specifico di APIGuard Assurance; è la conseguenza strutturale di qualsiasi approccio contract-driven, e la Sezione ?? ne discute le implicazioni pratiche.

La specifica come oracolo formale risponde alla domanda su *cosa* verificare; **P03 (CHANGE-ME)** risponde alla domanda su *come* il verificatore mantiene un comportamento stabile nel tempo. Si è scelto di vincolare ogni parametro di tuning operativo al file di configurazione, escludendo i literal decisionali dal codice, per una ragione che va oltre la pulizia implementativa: se il comportamento del sistema dipendesse dal codice sorgente oltre che dalla configurazione, due esecuzioni con lo stesso `config.yaml` sullo stesso target potrebbero produrre risultati diversi a seguito di un aggiornamento del tool, rendendo incomparabili assessment eseguiti su versioni differenti. L'insieme dei parametri configurabili comprende timeout, soglie di verifica, categorie di payload, strategie di test

abilitate e percorsi di output: nessuno di questi risiede nel codice come valore hardcoded. Il costo accettato è la rigidità della configurazione come interfaccia unica verso l'operatore, giustificata dal fatto che ogni deviazione da questo vincolo introduce un canale implicito di variabilità non tracciabile.

P28 (CHANGE-ME) traduce P03 (CHANGE-ME) in una garanzia verificabile: stesso `config.yaml`, stesso target, assenza di modifiche all'applicativo producono sempre lo stesso risultato. Il determinismo non è un obiettivo di ingegneria del software; è un prerequisito epistemologico. Un assessment non riproducibile non è un assessment: è un'osservazione irripetibile, il cui valore probatorio si esaurisce nel momento in cui viene prodotta. I risultati del Capitolo ?? hanno valore dimostrativo e non soltanto descrittivo esattamente perché P28 (CHANGE-ME) è verificato empiricamente su due run indipendenti, che hanno prodotto output byte-identici con un delta sul wall-clock di 0,32%, come documentato nella Sezione ?. Quella sezione illustra anche le condizioni precise in cui la garanzia vale e quelle in cui si indebolisce, completando la catena argomentativa aperta qui.

3.2 Architettura dell'Assessment Engine

L'architettura di *APIGuard Assurance* non è il prodotto di scelte accumulate per sedimentazione durante lo sviluppo, ma la realizzazione sistematica di sei proprietà strutturali che rispondono ciascuna a una frizione ingegneristica precisa. Questa sezione le presenta nell'ordine in cui emergono nella pipeline: dalla direzione delle dipendenze tra moduli all'ordinamento dell'esecuzione, dalla gestione dello stato alla persistenza delle evidenze forensi. Per ciascuna proprietà si esplicita il trade-off che la motiva e il costo accettato.

3.2.1 Flusso di Dipendenze Monodirezionale (P02)

Il grafo delle dipendenze tra moduli è strettamente aciclico e orientato in un'unica direzione: `core/` non importa mai da `tests/`; `tests/` e `external_tests/` importano esclusivamente da `core/`; `engine.py` importa da tutti e li coordina senza essere importato da nessuno. La regola è dichiarata nella docstring di apertura di ogni modulo del layer `core/` ed è verificabile con un singolo comando `grep` che, sul codebase di Milestone 1, produce zero risultati.

La conseguenza non è soltanto organizzativa. Modificare un test non può rompere `core/`: il grafo lo esclude per costruzione. Modificare `core/` può rompere i test, ma solo se l'interfaccia pubblica cambia, e la rottura è immediatamente visibile all'analisi statica. Su 90 file sorgente analizzati con `mypy` in modalità `strict`, il type checker non ha rilevato

nessuna violazione: da questo si inferisce che il contratto tra i componenti, espresso tramite type hints, è consistente lungo l'intera catena di dipendenze. Il costo accettato è una struttura gerarchica rigida, che impone a ogni nuovo modulo di collocarsi esplicitamente in uno dei layer. Tale rigidità è giustificata dal fatto che rende possibile, in una traiettoria futura, introdurre moduli di test contribuiti da terze parti tramite un meccanismo plugin: i moduli esterni rispettano la stessa gerarchia e non richiedono modifiche al core.

3.2.2 DAG-Based Scheduling e Orchestrazione Deterministica (P06)

L'ordine in cui i test vengono eseguiti non è fisso né casuale. Ogni test dichiara le proprie dipendenze tramite l'attributo `ClassVar depends_on`, una lista di `test_id` prerequisiti. Il `DAGScheduler` in `core/dag.py` costruisce un grafo orientato aciclico a partire da queste dichiarazioni e lo risolve tramite `graphlib.TopologicalSorter` della libreria standard Python. L'output è una sequenza di `ScheduledBatch`: i batch sono eseguiti in ordine topologico; all'interno di ogni batch, i test sono ordinati lessicograficamente per `test_id`, garantendo un ordinamento deterministico anche tra test privi di dipendenze reciproche.

Il legame con la riproducibilità (P28) è causale, non contingente. Senza un ordinamento deterministico garantito dalla struttura del grafo, due run identiche potrebbero schedulare gli stessi test in sequenze diverse, producendo stati del `TestContext` differenti e, di conseguenza, risultati non confrontabili. Il DAG non è un'ottimizzazione per la leggibilità del codice: è il meccanismo che rende P28 dimostrabile empiricamente, come verificato nella Sezione 5.3. Un ciclo di dipendenze è rilevato in Phase 4 tramite `DAGCycleError` e blocca l'avvio prima che qualsiasi test venga eseguito, evitando che stati inconsistenti si propaghino silenziosamente.

In Milestone 1, la topologia del DAG si articola in due fasi distinte: sedici test privi di dipendenze (Phase A, eseguiti per primi) e due test che dipendono da 1.1 (Phase B). L'asimmetria non è accidentale. I test perimetrali black-box non presuppongono stato accumulato e possono essere eseguiti immediatamente; i test grey-box che richiedono token JWT precedentemente acquisiti devono attendere che il test di autenticazione perimetrale (1.1) abbia confermato che l'endpoint di login è raggiungibile e funzionante. La struttura del grafo rende questo vincolo esplicito e verificabile, invece di affidarsi a convenzioni non enforced.

3.2.3 Split State e Immutabilità (P04)

Lo stato del sistema durante un assessment è diviso in due oggetti con caratteristiche deliberatamente opposte. Il `TargetContext`, definito con `model_config = {"frozen":`

`True}` in Pydantic v2, incapsula tutto ciò che si conosce del target prima dell'esecuzione: la specifica OpenAPI dereferenziata come `AttackSurface`, le credenziali configurate, i parametri di tuning per ogni test, l'adattatore gateway iniettato in Phase 3. Una volta costruito, non muta mai. Il `TestContext` è mutabile e accumula tutto ciò che si scopre o si produce durante l'esecuzione: i token JWT acquisiti, le risorse create sul target che dovranno essere rimosse al teardown, i dati calcolati da un test e consumati da test dipendenti.

L'immutabilità del `TargetContext` non è una scelta stilistica, ma la risposta a un problema preciso. Durante l'esecuzione sequenziale dei test, ciascuno con accesso alla rappresentazione del target, basterebbe una singola scrittura non sincronizzata per corrompere la base informativa da cui ogni verdetto dipende. Congelare lo stato a monte, prima che la pipeline si ramifichi, isola il problema alla radice: nessun test può alterare ciò che gli altri leggono. Il congelamento Pydantic applica questo vincolo a livello di tipo: qualsiasi tentativo di modifica solleva `ValidationError` al momento dell'assegnazione, prima che la corruzione si propaghi.

L'accesso allo stato mutabile del `TestContext` avviene tramite tre canali tipizzati distinti. Il canale token (`set_token`, `get_token`, `has_token`) è indicizzato per role constant, non per identificatore di test: le chiavi sono `ROLE_ADMIN`, `ROLE_USER_A`, `ROLE_USER_B`. Il canale teardown (`register_resource_for_teardown`, `drain_resources`) mantiene un registro LIFO delle risorse create sul target, garantendo che il cleanup avvenga nell'ordine inverso della creazione. Il canale shared data (`set_shared`, `get_shared`) è il meccanismo general-purpose per dati calcolati da un test e consumati da test dipendenti, con una convenzione di naming `"{test_id}.{data_name}"` che rende esplicito il test produttore. L'assenza di un dizionario libero a chiavi arbitrarie è intenzionale: un `get_token` che restituisce `None` è una condizione gestita esplicitamente dal chiamante, non un `KeyError` silenzioso che si propaga altrove.

Il costo accettato è la rigidità del `TargetContext`. Una volta costruito, non si aggiorna: qualsiasi informazione scoperta a runtime appartiene concettualmente allo stato dell'assessment, non allo stato del target, e trova il proprio posto nel `TestContext`. Questo confine non è sempre intuitivo, ma la sua esplicitezza è la garanzia che rende P28 dimostrabile senza ambiguità.

3.2.4 Gateway-Agnostic Adapter (P18)

I test `WHITE_BOX` interrogano il piano di configurazione del gateway per verificare parametri non osservabili dall'esterno: timeout configurati sui servizi, presenza e parametrizzazione dei plugin di circuit breaking, pattern delle credenziali nelle variabili d'ambiente. Questa logica non può essere separata dall'implementazione concreta del gateway senza un'astrazione esplicita; costruirla direttamente nei test violerebbe sia P01

(i test conterrebbero riferimenti Kong-specifici) sia P02 (i test dovrebbero importare la logica di connessione all'Admin API).

Il `BaseGatewayAdapter`, definito come Abstract Base Class in `src/core/gateway/base.py`, espone un'interfaccia uniforme read-only: `check_connectivity()`, `get_routes()`, `get_plugins()`, `get_services()`, `get_upstreams()`, `get_plugin_by_name()` e `get_status()`. L'implementazione concreta `KongGatewayAdapter` realizza questa interfaccia interrogando la Admin API di Kong DB-less tramite chiamate HTTP standard. La selezione a runtime avviene tramite il campo `gateway_adapter` nel `config.yaml`: il motore istanzia il tipo corretto in Phase 3, verifica la raggiungibilità tramite `check_connectivity()`, e inietta l'istanza in `TargetContext.gateway`. Se `admin_api_url` non è configurato, `target.gateway` rimane `None` e tutti i test `WHITE_BOX` transitano automaticamente a `SKIP` con motivo esplicito, senza errori a cascata.

La collocazione dell'adapter in `core/gateway/`, e non in `connectors/`, riflette una distinzione semantica precisa. I connector wrappano tool esterni per la raccolta di dati di sicurezza sul target applicativo; l'adapter interroga l'infrastruttura di controllo del gateway come estensione della fase di discovery. I test `WHITE_BOX` accedono all'adapter tramite `target.gateway`, indipendentemente dal tipo concreto sottostante: aggiungere supporto per un secondo gateway richiede una nuova sottoclasse di `BaseGatewayAdapter` e una riga nel dispatcher dell'engine, senza modificare nessuno dei test esistenti.

Il limite intrinseco di questa astrazione, vale a dire la tensione tra la generalità dell'interfaccia e la profondità delle verifiche vendor-specific che essa non può esprimere, viene discusso nella Sezione 6.1.

3.2.5 Streaming Evidence Store (P11)

Ogni finding prodotto dall'assessment deve essere corredato da evidenza HTTP dimostrabile: la request completa, la response completa, e un identificatore univoco che consenta di rintracciare il record nell'archivio forense. Questo requisito impone una scelta sulla gestione della capacità dell'archivio. Con il design iniziale basato su `collections.deque(maxlen=100)`, i test che producevano più di cento record FAIL, come il test 1.1 su un target con numerosi endpoint documentati, espellevano silenziosamente i record più vecchi dal buffer. I `Finding` prodotti contenevano valori `evidence_ref` che puntavano a record inesistenti nell'archivio: audit trail corrotto senza errori visibili e senza alcun modo di distinguere un finding con evidenza valida da uno senza.

L'architettura JSONL elimina questa classe di bug strutturalmente. Per ogni test, al momento dell'invocazione di `store.begin_test(test_id)`, viene aperto un file `.jsonl` dedicato in `outputs/evidence_tmp/`. Ogni record FAIL o *pinned* viene scritto immediatamente con flush. Al termine del test, `store.end_test()` chiude il file. In Pha-

se 7, `store.merge_and_finalize()` legge tutti i file JSONL, li ordina cronologicamente per `timestamp_utc` e produce il file `evidence.json` finale con struttura envelope. La directory temporanea viene rimossa al termine dell'operazione.

Due proprietà emergono da questa architettura senza richiedere logica aggiuntiva. La prima è la capacità illimitata: nessun record può essere espulso, indipendentemente dal volume prodotto da un singolo test. La seconda è la resilienza ai crash: i file JSONL in `evidence_tmp/` sopravvivono a un'interruzione del processo tra Phase 5 e Phase 7, rendendo l'archivio parzialmente recuperabile per ispezione manuale. Il costo accettato è un overhead di I/O superiore rispetto al buffer in memoria, giustificato dalla non-repudiabilità di ogni finding e dalla necessità che l'audit trail sia affidabile anche nelle condizioni operative più onerose.

3.2.6 Zero External State Dependency (P34)

Il tool non apre connessioni verso database, message broker, cache distribuite o qualsiasi altro servizio di stato esterno durante l'esecuzione dell'assessment. Tutto lo stato mutabile vive in memoria di processo nel `TestContext`; tutto lo stato persistente è scritto in file locali nella directory configurata tramite `output.directory`. Le uniche connessioni di rete aperte durante un run sono verso il target API, tramite `SecurityClient`, e, quando configurata, verso la Admin API del gateway.

Questo vincolo non è una scelta di semplicità operativa. È il prerequisito diretto dell'affidabilità dei semantic exit codes (P17). Un tool con dipendenze di stato esterno non può garantire che il proprio exit code rifletta fedelmente l'esito dell'assessment senza service readiness check preventivi: se il database che persiste i risultati è irraggiungibile, l'exit code potrebbe segnalare un errore infrastrutturale invece di una vulnerabilità nel target, rendendo il tool inutilizzabile come quality gate automatizzato. Di conseguenza, ogni exit code prodotto da *APIGuard Assurance* ha un significato non ambiguo, indipendente dallo stato dell'infrastruttura circostante. Ne deriva che il tool è deployabile in pipeline CI/CD senza overhead di provisioning: è sufficiente disporre di Python 3.11 e della connettività di rete verso il target.

3.3 Domini di Sicurezza e Box Gradient Applicato

La transizione dalla modellazione teorica delle minacce, discussa nel Capitolo ??, all'orchestrazione operativa dei test richiede un criterio di partizione rigoroso. L'eterogeneità delle vulnerabilità semantiche impone di frammentare l'analisi in aree di indagine logicamente isolate. Per rispondere a questa necessità, si è optato per strutturare l'architettura delle verifiche secondo una tassonomia originale articolata in otto Domini di Assurance:

- **Domain-0 (API Discovery):** Identificazione della superficie d'attacco effettiva, inclusa la rilevazione di endpoint ombra (*Shadow API*) e l'enforcement delle policy di negazione di default.
- **Domain-1 (Identity & Authentication):** Verifica dei meccanismi di riconoscimento del chiamante, della solidità dei protocolli di trasporto delle credenziali e dei processi di revoca dei JWT.
- **Domain-2 (Authorization):** Collaudo delle logiche di controllo degli accessi, finalizzato a isolare difetti di segregazione orizzontale e verticale (RBAC).
- **Domain-3 (Data Integrity):** Validazione della coerenza strutturale dei payload e della corretta implementazione delle firme crittografiche (es. HMAC).
- **Domain-4 (Availability & Resilience):** Valutazione delle difese contro l'esaurimento delle risorse, tramite l'audit dei *circuit breaker* e delle soglie di *rate limiting*.
- **Domain-5 (Observability):** Analisi della qualità e della tempestività della telemetria generata dal sistema (dominio architetturalmente predisposto per la Milestone 2).
- **Domain-6 (Configuration & Hardening):** Esame del livello di indurimento dell'infrastruttura sottostante, con particolare focus sulle policy del gateway e sugli header di sicurezza HTTP.
- **Domain-7 (Business Logic & Sensitive Flows):** Ricerca di falle semantiche complesse, quali l'elusione dei vincoli di processo e le vulnerabilità di tipo SSRF.

Si è stabilito che questa classificazione non debba costituire una mera convenzione per l'organizzazione del codice sorgente, bensì debba riflettere una rigida gerarchia di fiducia. L'assunto architetturale soggiacente è che l'interrogazione di un livello di sicurezza superiore sia concettualmente invalida qualora i vincoli del livello inferiore non risultino garantiti. È metodologicamente scorretto testare i meccanismi di autorizzazione (Domain-2) senza prima aver certificato l'impenetrabilità dei protocolli di autenticazione (Domain-1), così come risulta impossibile valutare l'autenticazione senza una mappa verificata della superficie esposta (Domain-0). Questa ferrea sequenzialità logica funge da preconditione per la costruzione del grafo aciclico discusso nella Sezione precedente.

Tale impostazione gerarchica trova il suo naturale complemento nell'applicazione dinamica del *Box Gradient* (D3.P3). Piuttosto che vincolare il framework a un unico paradigma di visibilità, si è optato per mappare strutturalmente il livello di accesso (Black, Grey e White Box) al grado di priorità (P0-P3) della garanzia indagata.

I test con priorità P0 operano sistematicamente in regime di *Black-Box*: simulano un attaccante esterno, privo di credenziali e di privilegi pregressi. L'adozione di questa postura di cecità iniziale è imperativa, in quanto rappresenta l'unico metodo empirico per ottenere un'evidenza di esposizione perimetrale che non risulti contaminata da autorizzazioni implicite.

I test di priorità P1 e P2 transitano invece verso un paradigma *Grey-Box*. Per ispezionare la corretta implementazione delle logiche di business e dei confini di segregazione, il motore necessita di esplorare uno stato autenticato plurimo, richiedendo la fornitura di token validi per almeno due ruoli applicativi distinti. In assenza di tali credenziali, i vettori d'attacco diretti contro i Domini 2 e 7 verrebbero respinti prematuramente dai filtri di identità, precludendo l'analisi della logica di business sottostante.

Infine, i test di priorità P3 adottano un approccio *White-Box*. Questa postura viene innescata esclusivamente quando il *target* concede al framework l'accesso diretto alla *Admin API* del gateway, permettendo l'estrazione statica e l'audit delle configurazioni di routing e dei plugin installati.

Abbracciare questa variazione dinamica della postura esplorativa impone una netta rinuncia alla semplicità operativa tipica dei fuzzer tradizionali. L'inevitabile conseguenza di tale frammentazione dei privilegi è il sensibile aumento della frizione in fase di configurazione, poiché si obbliga l'operatore a raccogliere e iniettare nel sistema materiale crittografico eterogeneo prima dell'avvio. Questo onere di setup iniziale rappresenta tuttavia un sacrificio calcolato: forzare l'architettura a mantenere un singolo e costante grado di visibilità avrebbe impedito, per definizione, di correlare l'esposizione perimetrale di una rotta non documentata con la misconfigurazione silente di un plugin di resilienza all'interno del medesimo ciclo di assessment.

Capitolo 4

Implementazione

4.1

Capitolo 5

Validazione sperimentale e risultati

5.1 Setup sperimentale

5.2 Risultati

Capitolo 6

Discussione e Sviluppi Futuri

6.1

Capitolo 7

Conclusioni

7.1 Conclusioni

7.2 Lavori futuri

Bibliografia